EMBEDDED SYSTEMS · COMPLETE PROJECT REPORT

Digital Clock

Built on a Bare ATmega328P-PU Microcontroller

From circuit calculations and breadboard prototyping through standalone chip migration, ISP bootloader flashing, soldering, and Fusion 360 snap-fit enclosure design.

ATmega328P-PU DIP-28 | 7-Segment Multiplexed | FDM 3D Printed Case

---

Project Documentation Report

Department of Electronics & Communication Engineering

June 25, 2026

Controller	ATmega328P-PU (standalone, no Arduino board in final product)
Display	4-digit multiplexed 7-segment (HH:MM and MM:SS modes)
Programmer	Arduino Uno configured as AVR ISP
Enclosure	Fusion 360 CAD, snap-fit two-shell, FDM 3D printed
Source	github.com/Bheemrajpaidipelli/IITH-Research-May

Abstract

This document covers the complete engineering lifecycle of a standalone digital clock assembled around the bare **ATmega328P-PU** microcontroller — the same silicon at the heart of the Arduino Uno, deployed here without the development board. The workflow is treated as a miniature product-development exercise: requirements translated directly into component selections, every passive value derived from first principles (crystal load capacitance, segment current-limiting resistors, decoupling and reset networks), logic first proven on an Arduino Uno breadboard, then ported to the bare chip after bootloader and fuse programming via a second Uno acting as an AVR ISP. Final assembly on a perforated dot board is followed by a custom **snap-fit enclosure** modelled in Autodesk Fusion 360 and printed in PLA on an FDM printer. Each engineering decision is backed by the short calculation that produced it.

Keywords: ATmega328P-PU · AVR In-System Programming · 7-Segment Multiplexing · Quartz Crystal Oscillator · Bootloader Fuse Burning · Fusion 360 · Snap-Fit Design · FDM Printing

0 Contents

1 Introduction

1.1 Motivation and Context

Every Arduino-based beginner project raises the same latent question: *what remains if the blue development board is removed?* Underneath its USB bridge chip, linear regulator, and pin headers, an Arduino Uno is a breakout board for a single IC — the **ATmega328P**. This project was conceived to answer that question deliberately, by rebuilding a fully-featured digital clock around the bare chip and documenting every engineering step that the development board normally handles automatically.

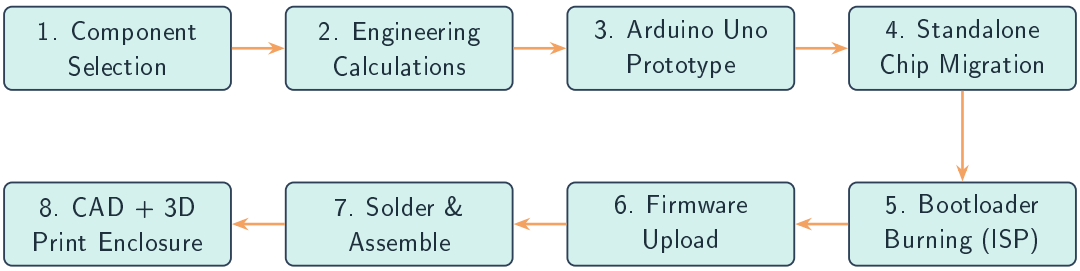
1.2 Project Objective

[i] Objective

Design and implement a **standalone digital clock** using the bare **ATmega328P-PU**, demonstrating the complete embedded product pipeline from component selection through enclosure fabrication, with every design choice backed by a supporting calculation.

1.3 Eight-Phase Development Workflow

The project was split into eight sequential phases. Nothing moves to the next phase without the current one being verified — a discipline that mirrors industrial embedded development practice.



[*] Why prototype on the Arduino first?

Using the Arduino Uno for early firmware iterations means a wiring error or logic bug costs a re-upload rather than a desoldering session. Only after full validation on the Uno does the design migrate to the bare ATmega328P-PU, where changes are considerably more involved.

2 Feature Set and User Interface

2.1 Display Modes

The firmware supports two modes selectable at runtime:

Mode	Display Format	Example
Normal (default)	Hours : Minutes	**14:35**
Seconds	Minutes : Seconds	**35:27**

After the Seconds mode timeout, the firmware automatically reverts to Normal mode so the clock can never become stuck.

2.2 Three-Button Control Scheme

Internal pull-up resistors are enabled in firmware for all button pins. Each button input therefore idles at logic HIGH; a keypress pulls it to LOW — no external resistors needed.

Button	Label	Action
B1	Hour	Increments the hours field; wraps 23 → 0
B2	Minute	Increments the minutes field; wraps 59 → 0
B3	Start/Mode	Starts a newly set time; long press enters Seconds mode

3 System Architecture

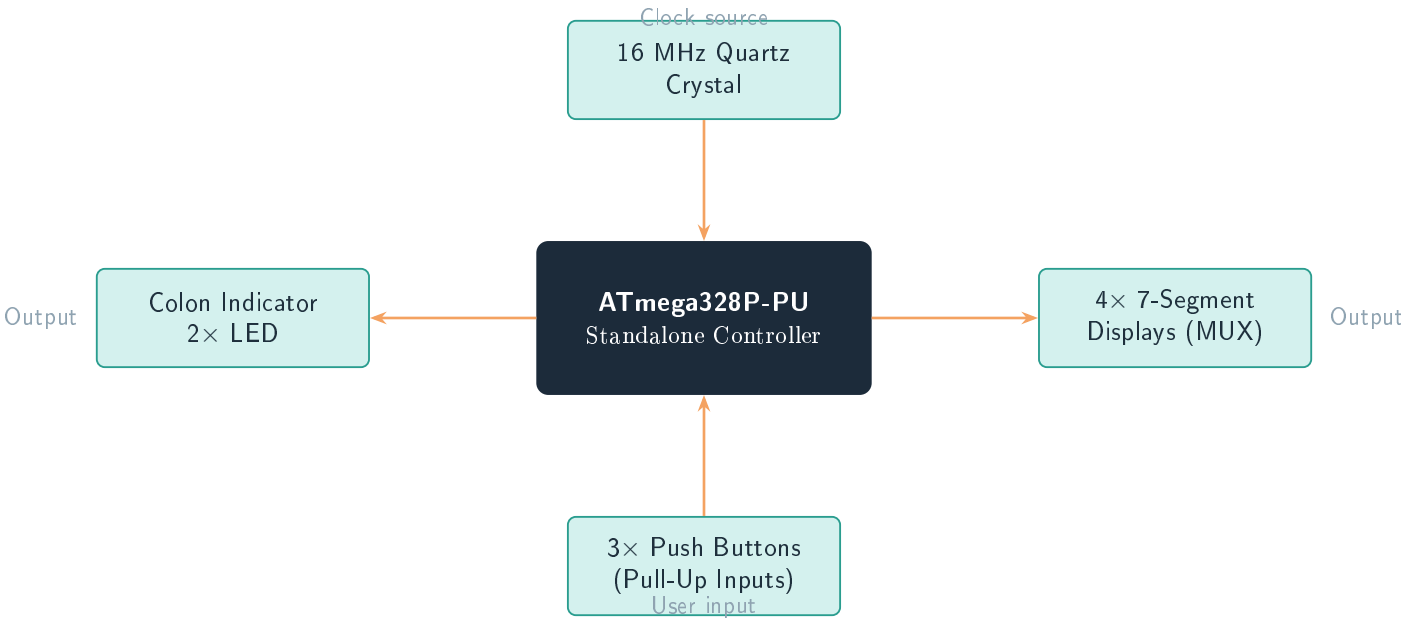


Figure 1. Top-level system block diagram. The ATmega328P-PU centralises all signal routing; no external driver ICs are required.

[i] No external driver ICs needed

The complete interface — 7 segment lines, 4 digit selects, 2 colon LEDs, 3 buttons — uses 14 GPIO lines, well within the ATmega328P-PU's 23 usable pins. The microcontroller drives everything directly.

4 Component Selection and Bill of Materials

4.1 Selection Rationale

Requirement	Component	Engineering Rationale
Processing core	ATmega328P-PU (DIP-28)	Identical silicon to the Arduino Uno; available for ∼80% less as a bare chip.
System clock	16 MHz quartz crystal	Arduino timing libraries require exactly 16 MHz; the internal RC oscillator is too inaccurate for reliable second-counting.
Crystal loading	2× 22 pF ceramic caps	Sets effective load capacitance inside the crystal's specified range (Section 5).
Supply bypass	2× 100 nF ceramic caps	Suppresses switching-induced voltage spikes on VCC and AVCC.
Reset stability	10 kΩ resistor	Holds `RESET` firmly HIGH so capacitive or inductive noise cannot trigger a spurious reset.
Time display	4× common-anode 7-segment	Multiplexing-friendly; inexpensive; good brightness at 10 mA.
Segment limiting	330 Ω resistors	Ohm's Law gives ≥300 Ω; nearest E12 value above that is 330 Ω.
Colon separator	2× LED + 330 Ω	Cheaper than a fifth display digit; unambiguous separator.
User input	3× tactile switches	Minimum controls for set/start/mode; internal pull-ups avoid external resistors.
Programmer	Arduino Uno as ISP	Reuses existing hardware; no dedicated USBasp needed.
Power	5 V USB module	Compatible with any USB charger or power bank.
Enclosure	Fusion 360 / FDM PLA	Snap-fit shell gives a finished product appearance without screws.

Table 1. Component selection rationale.

4.2 Bill of Materials

Component	Specification	Qty	Purpose
ATmega328P-PU	DIP-28	1	Main controller
Crystal	16 MHz HC-49	1	System clock
Ceramic cap	22 pF	2	Crystal load
Ceramic cap	100 nF	2	VCC/AVCC decoupling
Resistor	10 kΩ	1	RESET pull-up
Resistor	330 Ω	9	Segments (7) + colon LEDs (2)
7-Segment display	Common anode	4	Time display
LED	3/5 mm	2	Colon indicator
Tactile switch	6 mm THT	3	User input
DIP-28 socket	0.3" pitch	1	Chip protection / replaceability
USB power module	5 V regulated	1	Power supply
Dot board	Standard	1	Permanent assembly substrate
PLA filament	1.75 mm	≈1 spool	Enclosure shells

5 Engineering Calculations

5.1 Crystal Load Capacitance

The ATmega328P-PU requires the quartz crystal's load capacitance to be met by two external capacitors, one on each oscillator pin, both returned to ground.

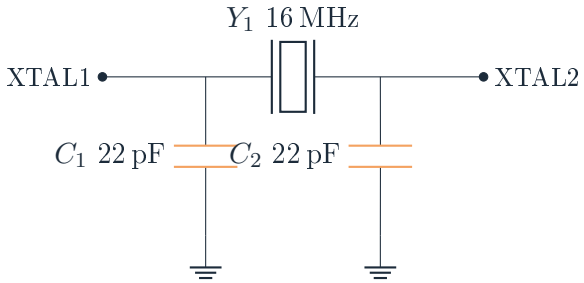


Figure 2. 16 MHz crystal oscillator network with 22 pF load capacitors.

[=] Load Capacitance Derivation

$$C_L = \frac{C_1 C_2}{C_1 + C_2} + C_{\text{stray}} = \frac{22 \times 22}{22 + 22} + C_{\text{stray}} = 11\,\text{pF} + 4\text{–}5\,\text{pF} \approx 15\text{–}16\,\text{pF}$$

This falls squarely within the 12–22 pF load capacitance window of standard AVR-grade 16 MHz crystals, confirming 22 pF as the correct capacitor value.

5.2 Decoupling Capacitor Placement

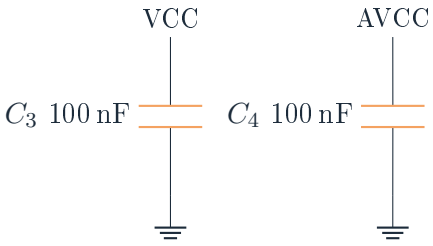


Figure 3. 100 nF decoupling capacitors on VCC and AVCC, placed as close to the chip as possible.

[*] Why 100 nF specifically?

Each time the multiplexer switches a digit, the AVR draws a brief current spike. Without decoupling, this manifests as a voltage dip on the supply rail that can trigger a spurious reset. A 100 nF ceramic capacitor, with its low ESR and ESL at digital switching frequencies, acts as a local charge reservoir that absorbs the spike before it propagates along the power trace.

5.3 RESET Pull-Up Network

The RESET pin is active-low. An unconnected RESET is a floating node that noise alone can pull below the reset-threshold voltage. A 10 kΩ resistor to 5 V holds the pin firmly HIGH during normal operation:

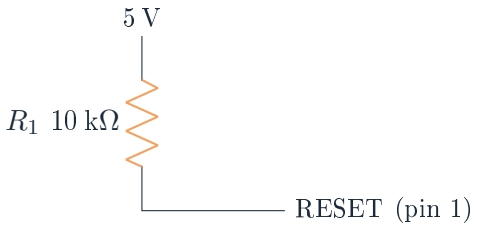


Figure 4. External RESET pull-up — holds the reset line at a stable 5 V.

5.4 Segment Current-Limiting Resistors

Figure 5. Current-limiting resistor in series with one 7-segment LED.

[=] Ohm's Law — Segment Resistor

$$R = \frac{V_{CC} - V_f}{I_{seg}} = \frac{5\,\text{V} - 2\,\text{V}}{10\,\text{mA}} = 300\,\Omega \quad \Rightarrow \quad \boxed{R_{chosen} = 330\,\Omega \quad \text{(nearest E12 value above } 300\,\Omega\text{)}}$$

The same 330 Ω value is applied to all seven segment lines (a–g) and to the two colon LEDs since they share identical forward-voltage characteristics.

[*] Multiplexing reduces average current

With only one digit active at any instant, the peak current of $7 \times 10\,\text{mA} = 70\,\text{mA}$ (a fully-lit "8") is not continuous — the average per pin over a full refresh cycle is a small fraction of that peak. In a future PCB revision, NPN/PNP transistors on the common (digit-select) lines would formally keep each AVR GPIO within its 20 mA continuous rating.

6 Phase I — Breadboard Prototype on Arduino Uno

6.1 Purpose

Every display and button interaction was validated on an Arduino Uno before any bare-chip work began. Mistakes at this stage cost only a re-upload.

6.2 Pin Allocation

Arduino Pins	Connected To	Role
D2–D8	Segments a, b, c, d, e, f, g	Shared segment bus for all four digits
D9–D12	Digit 1–4 common lines	Multiplexer digit select
A0–A2	Buttons 1, 2, 3	Hour / Minute / Start-Mode inputs

6.3 Multiplexing Principle

Driving four independent digits would require $4 \times 7 = 28$ GPIO pins. Multiplexing reduces this to 7 shared segment lines plus 4 digit-select lines by activating only one digit at a time, cycling fast enough that the eye perceives a steady display (persistence of vision threshold ≈50–60 Hz).

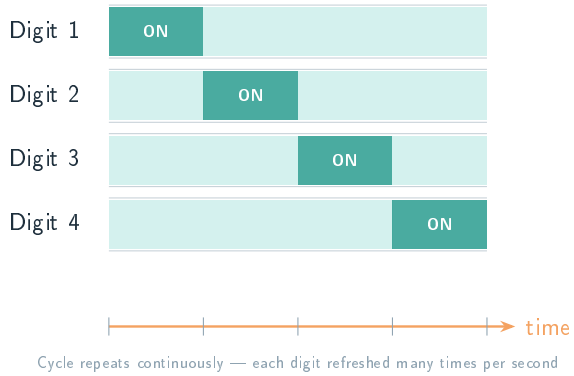


Figure 6. Multiplexing timing: only one digit is physically active at any moment.

6.4 Validation Checklist

Check	Result
Segment-to-physical mapping correct for all digits	✓ Pass
No flicker at chosen refresh rate	✓ Pass
No ghosting between adjacent digits	✓ Pass
Hour/minute increment with correct roll-over	✓ Pass
Seconds mode toggle and timeout	✓ Pass
Accurate timekeeping over extended soak	✓ Pass

7 Phase II — Migration to Standalone ATmega328P-PU

7.1 Chip Specification Summary

Parameter	Value
Supply voltage	5 V
Clock speed	16 MHz (external crystal)
Flash	32 KB
SRAM	2 KB
EEPROM	1 KB
Usable GPIO	23 pins
Package	PDIP-28

7.2 DIP-28 Pinout Reference

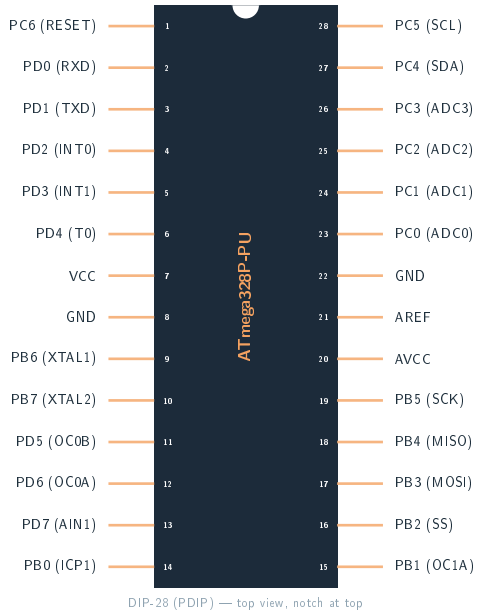


Figure 7. ATmega328P-PU PDIP-28 pinout (top view). Pin 1 is adjacent to the notch.

[*] Pre-allocated pins

Before any GPIO is assigned to the display or buttons, five pins are already consumed: VCC (7), GND (8), RESET (1), XTAL1 (9), XTAL2 (10). AVCC (20) and GND (22) also connect to the power rails, leaving exactly 23 usable GPIO lines — matching the datasheet figure.

7.3 Arduino Name to Physical Pin Cross-Reference

Arduino Pin	AVR Port Pin	Physical Pin #	Function in this Project
D2	PD2	4	Segment a
D3	PD3	5	Segment b
D4	PD4	6	Segment c
D5	PD5	11	Segment d
D6	PD6	12	Segment e
D7	PD7	13	Segment f
D8	PB0	14	Segment g
D9	PB1	15	Digit 1 select
D10	PB2	16	Digit 2 select
D11	PB3	17	Digit 3 select
D12	PB4	18	Digit 4 select
A0	PC0	23	Button 1 (Hour)
A1	PC1	24	Button 2 (Minute)
A2	PC2	25	Button 3 (Start/Mode)

8 Phase III — Bootloader Burning via Arduino as ISP

8.1 Why the Fresh Chip Needs a Bootloader

A factory ATmega328P-PU contains no Arduino bootloader and has fuse bits set for the internal 8 MHz oscillator — it is generic silicon. Before it will accept Arduino IDE sketches, the bootloader must be written and the fuse bits reconfigured for the 16 MHz external crystal. A second Arduino Uno, loaded with the `ArduinoISP` sketch, does both jobs over the SPI-based ISP interface.

8.2 ISP Wiring

Programmer Uno Pin	Target ATmega328P-PU Pin
D10	RESET (pin 1)
D11	MOSI (pin 17)
D12	MISO (pin 18)
D13	SCK (pin 19)
5 V	VCC (pin 7)
GND	GND (pin 8)

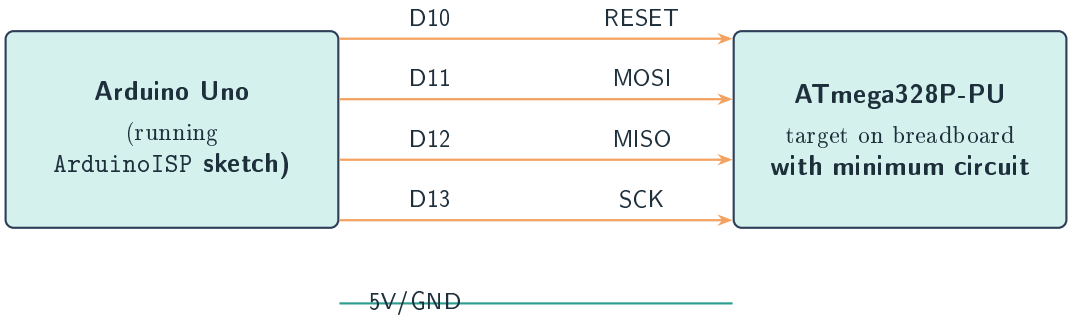


Figure 8. Arduino-as-ISP connection diagram used for both bootloader burning and firmware upload.

8.3 Burn Procedure

1. Upload **File** → **Examples** → **11.ArduinoISP** → **ArduinoISP** to the programmer Uno.
2. Wire programmer to target as per the table above; verify the target chip's crystal and reset network are in place.
3. In the IDE set **Board** = **Arduino Uno** and **Programmer** = **Arduino as ISP**.
4. Click **Tools** → **Burn Bootloader** and wait for the "Done burning bootloader" confirmation.

[!] Common Failure

If the crystal and 22 pF capacitors are not wired before burning, the operation will fail or hang — the ISP expects the target to run at the clock speed implied by the fuse bits being written.

9 Phase IV — Firmware Upload and Timekeeping Logic

9.1 Timekeeping Core

The clock derives hours, minutes, and seconds from the MCU's free-running millisecond counter — no external RTC IC is required.

Listing 1. Timekeeping core (simplified sketch excerpt).

```
unsigned long lastTick = 0;
int seconds = 0, minutes = 0, hours = 0;

void updateClock() {
  if (millis() - lastTick >= 1000) {    // one real second
    lastTick += 1000;                   // drift-compensated tick
    if (++seconds >= 60) { seconds = 0;
      if (++minutes >= 60) { minutes = 0;
        if (++hours   >= 24)   hours = 0;
      }
    }
  }
}
```

9.2 Uploading the Sketch to the Bare Chip

With the same ISP wiring as Section 8 in place, select **Sketch → Upload Using Programmer** (not the standard Upload button, which relies on a serial bootloader path that is bypassed by the programmer Uno).

[i] Full Source Code

The complete uploadable sketch is published at:
https://github.com/Bheemrajpaidipelli/IITH-Research-May/blob/main/Digital_clock_code

10 Phase V — Full-Circuit Standalone Validation

The entire standalone circuit — chip, crystal, decoupling, reset pull-up, all four displays, colon LEDs, and all three buttons — was re-assembled on a breadboard and powered from the 5 V USB module with no programmer Uno attached.

Test Case	Result
Chip runs without programmer attached	✓ Pass
Normal mode (HH:MM) displays correctly	✓ Pass
Seconds mode triggers and times out	✓ Pass
Hour and minute buttons with roll-over	✓ Pass
Colon LEDs visible and steady	✓ Pass
No flicker, ghosting, or dim digit	✓ Pass
RESET line measures 5 V during run	✓ Pass
No spurious resets over 30-min soak	✓ Pass

11 Phase VI — Soldering and Permanent Assembly

11.1 Soldering Discipline

- Keep crystal leads and the two 22 pF caps as **short as possible** to minimise stray inductance and capacitance.
- Verify **LED polarity** before soldering — incorrect orientation produces a dim or dead colon indicator.
- Check **chip orientation** (notch / pin-1 mark) before seating in the socket.
- Inspect every joint under magnification for **solder bridges** across DIP-28 pins.
- Power the board *without* the chip to verify 5 V and GND first; then insert the chip.

11.2 Soldered Board

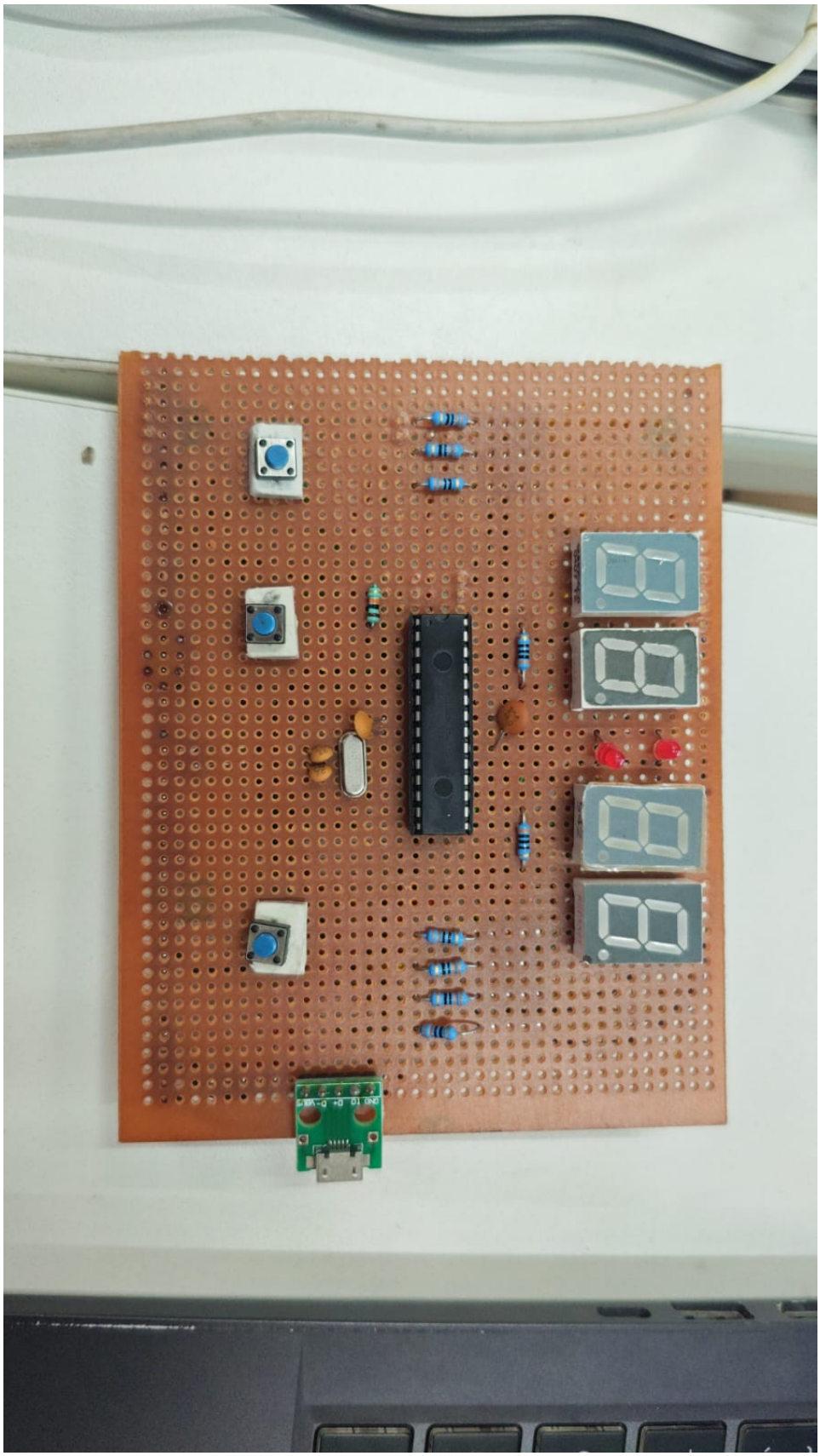


Figure 9. Completed soldered board: ATmega328P-PU in a DIP socket (centre), 16 MHz crystal and load capacitors (left of chip), four 7-segment displays (right), three user buttons (left column), resistors, colon LEDs, and USB power module (bottom).

11.3 Post-Solder Bring-Up

Check	Result
No solder bridges, no cold joints (visual)	✓ Pass
VCC/GND continuity on all power runs	✓ Pass
5 V verified before chip insertion	✓ Pass
Board functions identically to breadboard tests	✓ Pass

12 Phase VII — Enclosure Design in Fusion 360

12.1 Design Goals

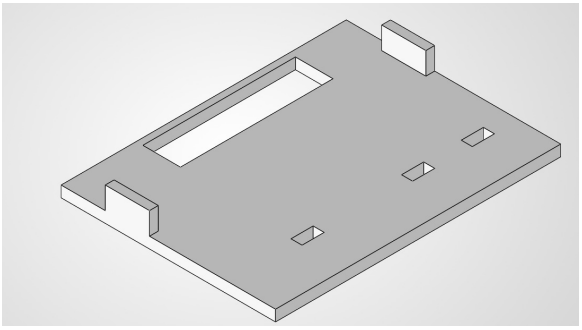
The two-shell enclosure was modelled to:

- Protect the soldered board and prevent accidental short-circuits;
- Provide a rectangular window for the display pair;
- Align cut-outs to the three push buttons;
- A USB port cut-out for power access;
- Assemble and disassemble **without screws**, using a snap-fit joint.

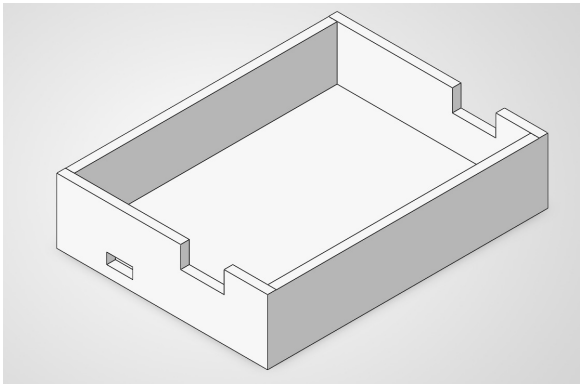
12.2 Snap-Fit Mechanism

Cantilever hooks moulded into the bottom shell flex past matching lips on the inside wall of the top shell, then spring back to lock. Tolerance is typically 0.2–0.3 mm clearance between hook and lip to balance secure engagement against hook-fracture risk.

12.3 CAD Renders



(a) Top shell — display window and button cut-outs.



(b) Bottom shell — internal snap-fit hooks and board standoffs.

Figure 10. Fusion 360 CAD renders of both enclosure shells.

[*] Snap-fit versus screws

A screw-fastened shell requires threaded bosses or self-tapping holes and leaves visible fastener heads. The snap-fit design assembles with a firm press and disassembles with a thin tool inserted into the seam — the exterior is hardware-free and the PLA material has sufficient flex to survive many assembly cycles if printed with the hooks oriented in the strongest layer direction.

13 Phase VIII — Slicing and 3D Printing

13.1 CAD to Print Pipeline

13.2 Key Slicing Decisions

Parameter	Rationale
Layer height	Finer layers improve exterior surface quality at the cost of print time.
Infill	Moderate infill sufficient — shells are not structural load-bearing parts.
Orientation	Snap hooks printed with the flex direction aligned to the strongest layer axis.
Supports	Used under the display window lip and button bosses to prevent drooping.

14 Results and Outcome

[i] Project Outcome

Final device: A self-contained digital clock running on a bare ATmega328P-PU, powered from a 5 V USB supply, housed in a two-piece snap-fit PLA enclosure. It reliably shows HH:MM in Normal mode, switches to MM:SS in Seconds mode on demand, and accepts hour/minute/mode inputs through three buttons — all confirmed through systematic breadboard, standalone, and post-solder validation at every phase.

Complete pipeline:

Requirement analysis → Calculations → Arduino Uno prototype → Bare-chip migration → ISP bootloader & firmware → Standalone re-validation → Soldered assembly → Fusion 360 enclosure → FDM-printed final product.

15 Skills Demonstrated

- Embedded design with a bare microcontroller (no development board in final product)
- Passive component calculation: oscillator, decoupling, and reset networks
- 7-segment display multiplexing and persistence-of-vision driven firmware

- Arduino C/C++ programming with `millis()`-based scheduling
- AVR ISP programming and bootloader/fuse management
- Multi-stage hardware debugging (two breadboard iterations + post-solder)
- Through-hole soldering and permanent dot-board assembly
- Parametric CAD in Autodesk Fusion 360 with snap-fit mechanical design
- FDM slicing and 3D printing workflow (Creality Slicer)

16 Future Improvements

- **DS3231 RTC module** for battery-backed, drift-free timekeeping across power cycles.
- **Alarm and buzzer** using a spare GPIO pin and a piezo transducer.
- **Digit-driver transistors** to keep every AVR GPIO within its 20 mA continuous rating per the datasheet recommendation.
- **Custom PCB** fabrication to replace the dot board with a compact, reliable layout.
- **PWM brightness control** with an LDR for ambient-adaptive display dimming.
- **Temperature/date sub-mode** rotating in the same four-digit display.

17 Conclusion

This project treated a digital clock as an exercise in following a real embedded-product workflow rather than a shortcut to a working display. Removing the Arduino board forced every invisible service it provides — oscillator loading, reset conditioning, power bypassing, pin mapping, bootloader management — to be handled explicitly, with a supporting calculation or documented procedure for each. The result is both a functional clock and a complete worked example of the idea-to-enclosed-product pipeline that underlies professional embedded hardware development.

18 References

- Project Arduino Sketch: https://github.com/Bheemrajpaidipelli/IITH-Research-May/blob/main/Digital_clock_code
- ATmega328P-PU Datasheet — Microchip Technology / Atmel Corporation
- Arduino IDE built-in example: `File` → `Examples` → `11.ArduinoISP` → `ArduinoISP`
- Autodesk Fusion 360 — parametric CAD software used for enclosure modelling
- Creality Slicer — G-code generation software for FDM printing